

axodendron

Deterministic neuronal morphology analysis and rendering from SWC

v0.1.1

2026-08-23

MIT

Analyze and visualize neuronal morphologies from SWC files.

Eito Yoneyama

Axodendron validates, analyzes, transforms, exports, and renders neuronal morphology data in Typst. A deterministic Rust/WebAssembly core owns SWC parsing, topology, morphometrics, transformations, and SVG geometry; Typst owns document layout and native annotations.

Table of Contents

Getting Started	3	VIII.1 SWC-Compatible Geometry	17
I.1 Choosing a Validation Profile	4	VIII.2 General Affine Geometry	18
I.2 Cell Values	4	VIII.3 Selection and Extraction	18
Compatibility and Architecture	5	VIII.4 Rerooting and Pruning	18
Example Data, Attribution, and License .	6	VIII.5 Simplification	19
Validation and Provenance	8	VIII.6 Resampling	20
IV.1 Profiles in Practice	8	VIII.7 Deterministic Export	20
Morphometric Analysis	9	Topology Visualization	21
V.1 Domains and Topology	9	Topological Morphology Descriptor	22
V.2 Sections, Paths, and Tortuosity	10	Population Features	24
V.3 Branch and Strahler Order	10	Rendering	25
V.4 Radius-Dependent Metrics	11	XII.1 Projections	25
Versioned Metric System	12	XII.2 Geometry and Radius Modes	26
VI.1 Selection and Explicit Field		XII.3 Soma Modes	26
Conversion	12	XII.4 Type and Scalar Color	26
VI.2 Branch and Section Definitions . . .	13	XII.5 Fitting and Aspect Ratio	27
VI.3 Basic Versioned Metrics	14	Native Typst Annotations	28
VI.4 Spatial Metrics and Principal		XIII.1 CeTZ Leader Labels	29
Frames	14	XIII.2 Render Reports	30
Sholl Analysis	16	Publication Figure Guidance	32
Pure Transformations	17	Error Model and Resource Limits	33

API Reference	34
XVI.1 Loading and Inspection	34
XVI.2 Analysis	34
XVI.3 Population Tables	37
XVI.4 Selection and Transformations ..	38
XVI.5 Annotation Builders	39
XVI.6 Renderer	42
Result Schemas	47
XVII.1 Analysis Bundle	47
XVII.2 Metric Result	47
XVII.3 Principal Frame	47
XVII.4 TMD Result	47
XVII.5 Feature Table	47
XVII.6 Transform Result	48
XVII.7 Render Result	48
Limitations	49
License, Dependencies, and Data	
Citations	50
Index	51

Part I

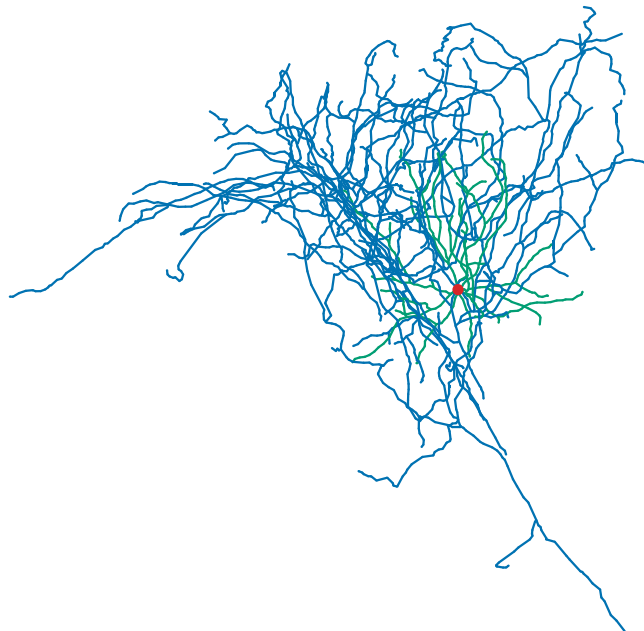
Getting Started

Import axodendron, read an SWC file as bytes, validate it, and pass the resulting immutable cell to `#render`. The filename in a user document is resolved by the calling document, not by the package.

```
#import "@preview/axodendron:0.1.1" as swc

#let cell = swc.load(
  read("Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc", encoding:
none),
  profile: "incf-strict",
)

#swc.render(cell, width: 120mm, height: 90mm)
```



Data: Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc NeuroMorpho.Org record 62495 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

The examples below assume `#import "@preview/axodendron:0.1.1" as swc`. Code snippets treat the directory containing the selected `.swc` file as the caller root. This manual keeps its Typst sources and data separate, so its executable setup reads the same files from `examples/data/`.

On Typst 0.15.0 and later, `#load` can receive `path("AA0109.CNG.swc")` directly. Axodendron reads the path inside the package call while retaining caller-relative resolution. This manual uses

`read(..., encoding: none)` because the minimum supported compiler is Typst 0.14.0 and the documentation toolchain is pinned to Typst 0.14.2.

SWC coordinates and radii are plain numbers in `cell.units`, conventionally micrometres. Typst lengths such as 120mm, 4pt, and 8pt control document layout only and are never mixed into morphology calculations.

I.1 Choosing a Validation Profile

Use profile: "incf-strict" for publication inputs expected to satisfy INCF SWC 1.0 ordering. It requires sequential positive IDs, parent-before-child order, a first root with parent -1, and one connected root. Use profile: "permissive" when a structurally valid archive contains arbitrary positive IDs, out-of-order rows, or a rooted forest that must be preserved exactly.

Neither profile repairs morphology. Duplicate IDs, missing parents, self-parenting, cycles, malformed rows, and non-finite geometry remain errors. Permissive mode relaxes ordering and connectedness only.

I.2 Cell Values

A successful load returns a dictionary containing `valid`, `diagnostics`, `fingerprint`, `source-fingerprint`, `node-count`, `units`, `metadata`, and a private canonical payload. The semantic fingerprint ignores harmless text formatting but changes when morphology semantics change. The source fingerprint tracks the exact input bytes.

```
#let cell = swc.load(read("AA0109.CNG.swc", encoding: none))

#table(
  columns: 2,
  [Valid], [#cell.valid],
  [Nodes], [#cell.node-count],
  [Units], [#cell.units],
  [Semantic fingerprint], [#cell.fingerprint],
)
```

Valid	true
Nodes	2172
Units	um
Semantic fingerprint	fnv1a64:b81103a39dfa984a

Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

Part II

Compatibility and Architecture

Surface	Minimum	Coverage
Typst package with string or bytes input	0.14.0	CI compiles the complete package and smoke suite on the minimum and latest configured Typst versions
Typst path(. . .) input	0.15.0	A version-gated smoke assertion loads a real file through path(. . .)
Rust workspace	1.85.0	Ubuntu minimum plus stable Rust on Ubuntu, macOS, and Windows
Detailed manual	0.14.2	Built reproducibly with just docs and the pinned Nixpkgs revision

The package is split into a format-independent Rust core, an SVG renderer, and a thin Typst minimal-protocol adapter. The WASM plugin is pure and stateless: it cannot access files, clocks, randomness, environment variables, or the network. Equal request bytes produce equal response bytes.

The wire boundary uses deterministic CBOR with `protocol_version: 2`. Morphology payload schema 2 serializes canonical arrays, provenance, and fingerprints; derived children, roots, components, lookup maps, and soma classification are rebuilt and validated on every decode.

Part III

Example Data, Attribution, and License

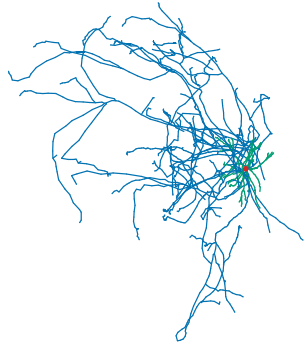
This manual uses four standardized real SWC reconstructions from NeuroMorpho.Org. They are distributed under [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)¹. Reuse requires attribution to the original study and NeuroMorpho.Org, RRID:SCR_002145. NeuroMorpho.Org also requests citation of its database publications, including Tecuatl, Ljungquist, and Ascoli (2024), [doi:10.1096/fba.2024-00048](https://doi.org/10.1096/fba.2024-00048)².

File	Record	Archive	Original study
AA0109.CNG.swc	85226 ³	MouseLight	10.1002/jnr.23978 ⁴ deposit ⁵
Nr5a1-Cre_Ai14-187777-05-02-01_491392821_m.kp1.swc	62390 ⁶	Allen Cell Types	10.1016/j.neuron.2015.02.022 ⁷
Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc	62495 ⁸	Allen Cell Types	10.1016/j.neuron.2015.02.022 ⁹
Vipr2-IRES2-Cre_Ai14-310513-05-02-01_637021223_m.CNG.swc	102520 ¹⁰	Allen Cell Types	10.1016/j.neuron.2015.02.022 ¹¹

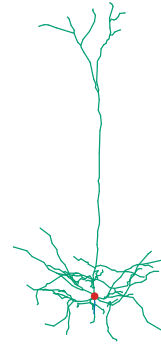
```
#grid(  
  columns: 2,  
  gutter: 5mm,  
  row-gutter: 5mm,  
  swc.render(aa, width: 65mm, height: 48.75mm),  
  swc.render(nr5a1, width: 65mm, height: 48.75mm),  
  swc.render(sst, width: 65mm, height: 48.75mm),  
  swc.render(vipr2, width: 65mm, height: 48.75mm),  
)
```

¹<https://creativecommons.org/licenses/by/4.0/>
²<https://doi.org/10.1096/fba.2024-00048>
³<https://neuromorpho.org/api/neuron/id/85226>
⁴<https://doi.org/10.1002/jnr.23978>
⁵<https://doi.org/10.25378/janelia.5526706>
⁶<https://neuromorpho.org/api/neuron/id/62390>
⁷<https://doi.org/10.1016/j.neuron.2015.02.022>
⁸<https://neuromorpho.org/api/neuron/id/62495>
⁹<https://doi.org/10.1016/j.neuron.2015.02.022>
¹⁰<https://neuromorpho.org/api/neuron/id/102520>
¹¹<https://doi.org/10.1016/j.neuron.2015.02.022>

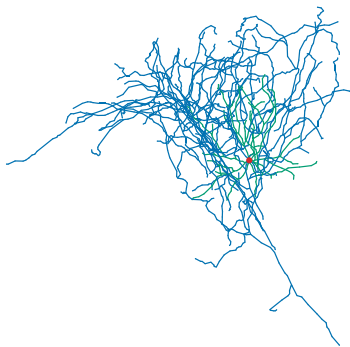
III Example Data, Attribution, and License



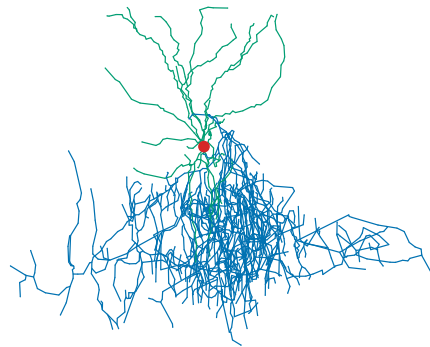
Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.



Data: Nr5a1-Cre_Ai14-187777-05-02-01_491392821_m.kp1.swc NeuroMorpho.Org record 62390 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.



Data: Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc NeuroMorpho.Org record 62495 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.



Data: IRES2-Cre_Ai14-310513-05-02-01_637021223_m.CNG.swc NeuroMorpho.Org record 102520 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

The real files are included only as attributed documentation and README examples. Their source URLs, SHA-256 checksums, archive names, original studies, and license are also recorded in `THIRD_PARTY_NOTICES.md`. The separate 30-case regression corpus remains an ignored, checksum-verified private cache and is not distributed.

Part IV

Validation and Provenance

`#diagnostics` returns every retained diagnostic as a dictionary with code, severity, message, optional line, optional column, and optional node-id. Set `(fail-on-error)` to false when a document should inspect invalid input instead of stopping immediately.

```
#let malformed = swc.from-text(  
  "1 1 0 0 0 2 -1\n1 3 0 5 0 1 1\n",  
  fail-on-error: false,  
)  
  
#for item in swc.diagnostics(malformed) [  
  #item.severity: #item.code - #item.message  
]
```

Severity	Code	Message
error	SWC_DUPLICATE_ID	node id 1 duplicates line 1
error	SWC_SELF_PARENT	node cannot be its own parent
warning	SWC_MULTIPLE_ROOTS	morphology has 2 roots; expected one
warning	SWC_DISCONNECTED_COMPONENT	morphology contains 2 disconnected components

`#metadata` exposes recognized header fields without applying them. Scale, shrinkage correction, species, region, and free-form comments remain provenance. Axodendron never changes coordinates or radii because a header suggests a correction.

IV.1 Profiles in Practice

`incf-strict` is appropriate for standardized NeuroMorpho.Org files and reproducible interchange. `permissive` is appropriate for structurally valid rooted forests or legacy tools that do not preserve row ordering. Exporting a single-root transformed cell through `#export-swc` produces sequential, topologically ordered IDs that can be loaded strictly again.

Part V

Morphometric Analysis

`#analyze` computes one versioned bundle containing a summary, topology, sections, tortuosity, and four node-aligned fields. The default neurites domain excludes type-1 soma nodes and every soma-incident edge; raw includes every encoded node and edge.

```
#let metrics = swc.analyze(cell)
#let summary = metrics.summary

#table(
  columns: 2,
  [Nodes], [#summary.node_count],
  [Cable length], [#summary.total_cable_length],
  [Branch points], [#summary.branch_point_count],
  [Terminals], [#summary.terminal_count],
)
```

Domain	neurites
Neurite nodes	2169
Cable length	90219.4 um
Branch points	119
Terminals	127
Sections	245
Maximum root path	6554.31 um

Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

V.1 Domains and Topology

Each non-soma node whose parent is excluded becomes an independent arbor root in the neurites domain. A node with more than one included child is a branch point; a node with no included child is terminal. `topology.node-ids`, `parent-ids`, `root-ids`, `terminal-ids`, `branch-point-ids`, and `component-ids` are aligned and deterministic.

Use domain: "raw" only when a measurement should follow the encoded graph convention, including soma nodes and soma connectors. The distinction matters when comparing cable length or path distance with external tools.

V.2 Sections, Paths, and Tortuosity

The default section-boundaries: "topology-and-type" starts or ends a section at roots, branch points, terminals, and SWC type changes. The transition edge belongs to the proximal section, so section lengths sum exactly to total cable length. "topology-only" ignores type changes.

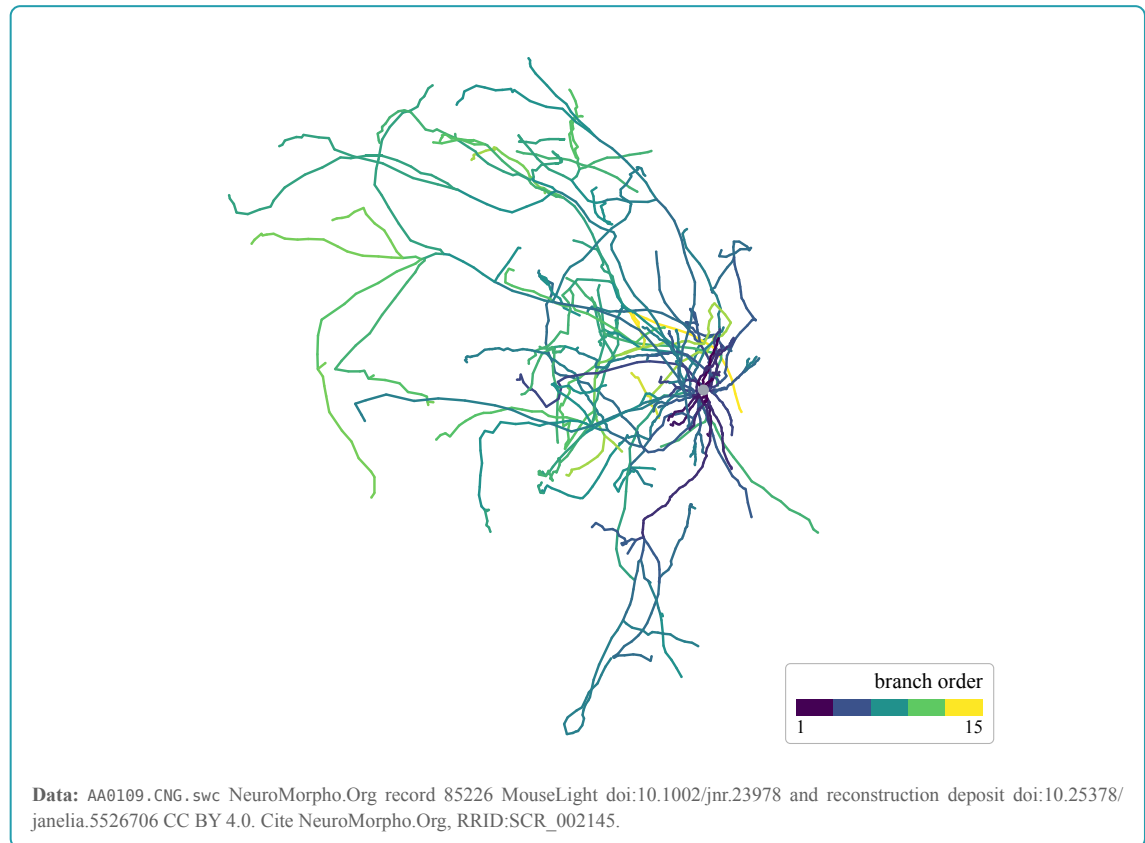
Root path length accumulates 3D Euclidean segment lengths from each arbor root. Radial distance is the straight-line 3D distance from that root. Section tortuosity is path length divided by endpoint distance; zero-span sections are excluded and counted separately.

V.3 Branch and Strahler Order

Centrifugal branch order is 1 at each arbor root and increments on children of a branch point. Strahler order is computed independently per arbor: terminals are 1, and a parent increments the maximum child order only when that maximum occurs at least twice.

```
#let metrics = swc.analyze(cell)

#swc.render(
  cell,
  color-by: metrics.branch_order,
  colormap: "viridis",
  minimum: 1,
  maximum: 20,
  color-bar: swc.color-bar(
    min: 1,
    max: 20,
    label: [branch order],
  ),
)
```



Node fields include name, node-ids, values, units, fingerprint, domain, and definition-version. Passing a field from another cell to `#render` is rejected instead of silently assigning values to the wrong geometry.

V.4 Radius-Dependent Metrics

Each neurite segment is modeled as an uncapped circular frustum with axial length L and endpoint radii r_0, r_1 :

$$\text{area} = \pi(r_0 + r_1)\sqrt{L^2 + (r_0 - r_1)^2}$$

$$\text{volume} = \pi L \frac{r_0^2 + r_0 r_1 + r_1^2}{3}$$

No end caps are added. A non-positive endpoint radius makes aggregate surface area and volume unavailable, and the offending node IDs are returned. Input radii are never repaired silently.

A single-point soma exposes sphere area and volume. A valid NeuroMorpho three-point soma exposes equivalent-sphere values and the encoded cylinder's lateral area and volume. Equal radii, endpoint distance, and endpoint opposition use a 1% scale-relative tolerance. Other soma encodings do not receive invented scalar metrics.

Part VI

Versioned Metric System

`#measure` complements the fixed `#analyze` bundle with individually versioned metrics. A request uses one common selection query, and each returned `MetricResult` records the stable metric ID, an independent definition version, all resolved parameters, morphology/topology/selection fingerprints, implementation provenance, a tagged data container, and explicit missing values.

The data tag is one of `morphology-metric`, `node-field`, `section-field`, or `bifurcation-field`. A bifurcation key records the branch node and the participating child sections, so pairwise values remain unambiguous at multifurcations. A section key is intentionally transient: it includes the topology fingerprint, selection fingerprint, section-definition revision, boundary policy, and proximal/distal node IDs. It must not be persisted as an identity independent of those fields.

Missing values are not encoded as an unexplained `none`. Every undefined entity carries a reason such as `non-binary-bifurcation`, `zero-length`, `non-positive-radius`, or `degenerate`, plus a detail string. Do not discard this information when aggregating results.

VI.1 Selection and Explicit Field Conversion

The common selection query combines domain, kinds, subtree roots, and explicit nodes. The result is always an induced forest: an original edge is included only when both endpoints survive. Topology-aware selectors return roots, branch points, terminals, soma nodes, or nodes constrained by centrifugal branch order or Strahler order under exactly the same query semantics. Orders are computed after applying the query, so they describe the selected induced forest rather than a hidden full-cell topology.

`#branch-order` and `#strahler-order` construct selectors with one exact positive order or an inclusive min/max interval. Their convenience wrappers `#branch-order-nodes` and `#strahler-order-nodes` return IDs directly. Exact and range constraints are mutually exclusive; zero, negative, and inverted ranges are rejected.

Section and bifurcation values are not node fields. Use `#field-to-nodes` with an explicit placement and collision reducer before passing them to `color-by`. The default reducer is `error`; this prevents pairwise bifurcation values from being silently collapsed at a multifurcation.

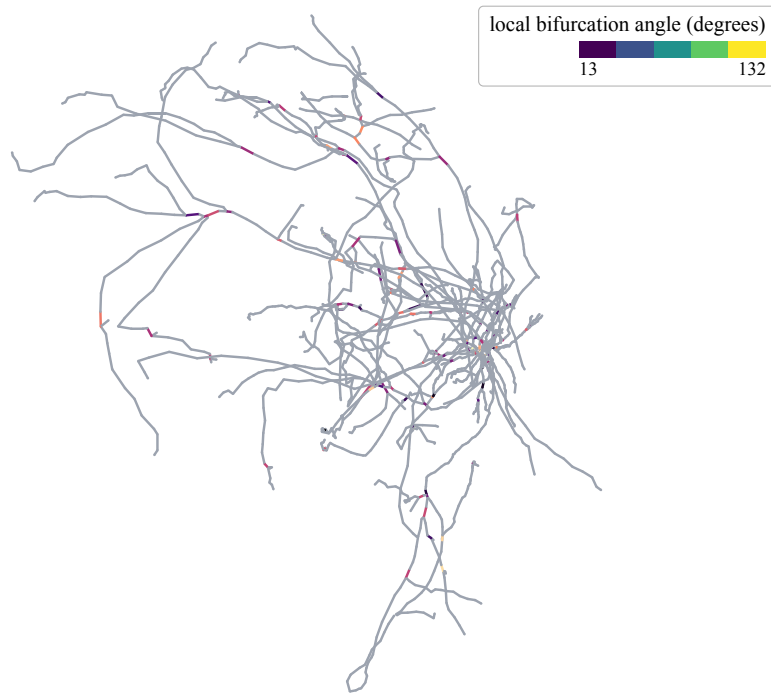
```
#let angle = swc.measure(  
  cell,  
  metrics: swc.metric(  
    "local-bifurcation-angle",  
    multifurcation: "pairwise",  
  ),  
)  
.first()  
#let at-branches = swc.field-to-nodes(  
  cell,
```

```

    field: angle,
    placement: "bifurcation-branch",
    reducer: "mean",
  )

#swc.render(cell, color-by: at-branches, colormap: "magma")

```



Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

VI.2 Branch and Section Definitions

Metric ID	Definition
local-bifurcation-angle	Angle between the first non-zero outgoing child-segment vectors.
remote-bifurcation-angle	Angle between vectors from the branch node to the distal endpoints of the outgoing sections.
sibling-ratio	$\frac{\min(d_1, d_2)}{\max(d_1, d_2)}$ for positive child diameters; the sampling rule is recorded.
partition-asymmetry-terminal	$\frac{ N_1 - N_2 }{N_1 + N_2 + 2}$ using selected-subtree terminal counts; two terminal children are defined as zero.
diameter-power-ratio	$\sum_i \frac{d_i^p}{d_0^p}$ for every outgoing child; p and diameter sampling are recorded.
rall-ratio	Alias of the diameter-power ratio with default $p = \frac{3}{2}$. A value of one satisfies that parameterized relation.
taper-rate	Signed slope of radius or diameter against cumulative section path length; ordinary least squares is the default and endpoint difference is optional.

Metric ID	Definition
segment-meander-angle	Turning angle between consecutive non-zero outward segment vectors: zero is locally straight and 180 degrees is reversal.

Angles are serialized in degrees on $[0, 180]$. Pairwise processing is the default for multifurcations and is recorded in the key; choose `multifurcation`: "exclude" when a binary-only definition is required. Sibling ratio follows the smaller-over-larger convention on $[0, 1]$. Diameter power ratios include all outgoing children and require positive parent and child diameters.

The taper result is signed: negative means narrowing from root toward tip. `taper-quantity`: "diameter" is the default, while "radius" is a distinct resolved definition. The fitted abscissa is cumulative three-dimensional path distance at the original SWC samples; no resampling or smoothing occurs.

NeuroM's `section_meander_angles` uses the supplementary opening-angle convention, where a straight continuation is π radians. Axodendron's turning angle is therefore 180 degrees minus the NeuroM opening angle after conversion to degrees. This convention transform and NeuroM's radians-to-degrees transform are recorded explicitly in the cross-validation fixture rather than described as exact definition equality.

VI.3 Basic Versioned Metrics

Metric IDs	Support	Definition
node-count, branch-point-count, terminal-count, section-count	Morphology	Counts in the selected induced forest.
total-cable-length, maximum-root-path-length	Morphology	Stable three-dimensional edge sum and maximum selected-root path distance.
neurite-surface-area, neurite-volume	Morphology	Uncapped-frustum aggregates; any non-positive endpoint radius yields explicit missing records rather than a partial total.
root-path-length, radial-distance	Node	Distance from each selected induced-arbor root.
branch-order, strahler-order	Node	Discrete topology fields on the selected forest.
section-length, section-contraction	Section	Section path length and endpoint distance divided by path length.

These metrics use the same tagged `MetricResult`, selection fingerprints, definition versions, missing reasons, and population aggregation rules as definition-sensitive branch metrics. The fixed `#analyze` bundle remains convenient for a broad report, while `#measure` is the comparable and provenance-bearing interface.

VI.4 Spatial Metrics and Principal Frames

Metric ID	Support and interpretation
centroid	Node mean or exact continuous cable-length centroid.
bounding-box	Axis-aligned minimum and maximum of selected node centers; radii are excluded.

Metric ID	Support and interpretation
principal-extents	Full min-to-max spans of selected node centers in a deterministic principal frame.
fractional-anisotropy	Fractional anisotropy of the three spatial covariance eigenvalues.
convex-hull-2d-area	Projected node-center hull area in a coordinate or principal plane.
convex-hull-3d-surface-area	Surface area of the three-dimensional node-center hull.
convex-hull-3d-volume	Volume of the three-dimensional node-center hull.
volume-density	Selected uncapped-frustum volume divided by the three-dimensional node-center hull volume.

Cable-length weighting analytically integrates the first and second moments of every selected line segment, rather than treating sample points as independent observations. Consequently, inserting collinear samples leaves the resulting centroid and covariance unchanged up to floating-point rounding. Node weighting is available when the sampling distribution itself is intentionally part of the statistic.

`#principal-frame` is a first-class result rather than a loose metric. It returns origin, centroid, ordered axes, covariance eigenvalues, extents, numerical rank, ambiguous-axis flags, and complete tolerance/sign/handedness provenance. Eigenvalues are ordered from largest to smallest. Axis signs use a deterministic farthest-projection rule with node-ID tie breaking, and the final basis is right-handed.

Repeated or nearly repeated eigenvalues make individual axes scientifically non-identifiable. Axodendron reports that degeneracy using $\varepsilon_a + \varepsilon_r \max(\text{lambda}_i, \text{lambda}_j, 1)$; it does not imply a biologically meaningful orientation merely because a reproducible numerical basis can be chosen. `#pca-align` rejects such a frame unless `allow-degenerate: true` is explicit.

Named render projections "principal-xy", "principal-xz", and "principal-yz" compute the default cable-weighted frame and project without modifying the morphology. Use `swc.pca-align` only when a transformed SWC-compatible coordinate set is actually required.

Part VII

Sholl Analysis

`#sholl` intersects original 3D segments with spheres. `#sholl-2d` first applies the chosen physical orthographic projection, then intersects the projected segments with circles. Display simplification is never used for either calculation.

```
#let result = swc.sholl(  
  cell,  
  radii: range(20, 220, step: 20),  
)  
  
#table(  
  columns: 2,  
  table.header([Radius], [Intersections]),  
  ..result.bins.map(bin => (bin.radius, bin.intersections)).flatten(),  
)
```

Radius (um)	Intersections
20	15
40	34
60	49
80	55
100	61
120	56
140	58
160	51
180	44
200	45

Data: Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc NeuroMorpho.Org record 62495 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

The default center is the represented soma center, or the sole root when no soma exists. Soma-free forests and disconnected soma subgraphs have no unique default; provide `<center>` or `<center-node>`. Intersections at segment parameter `t` in $(0, 1]$ count, so a shared vertex is counted once and a tangency counts once.

Part VIII

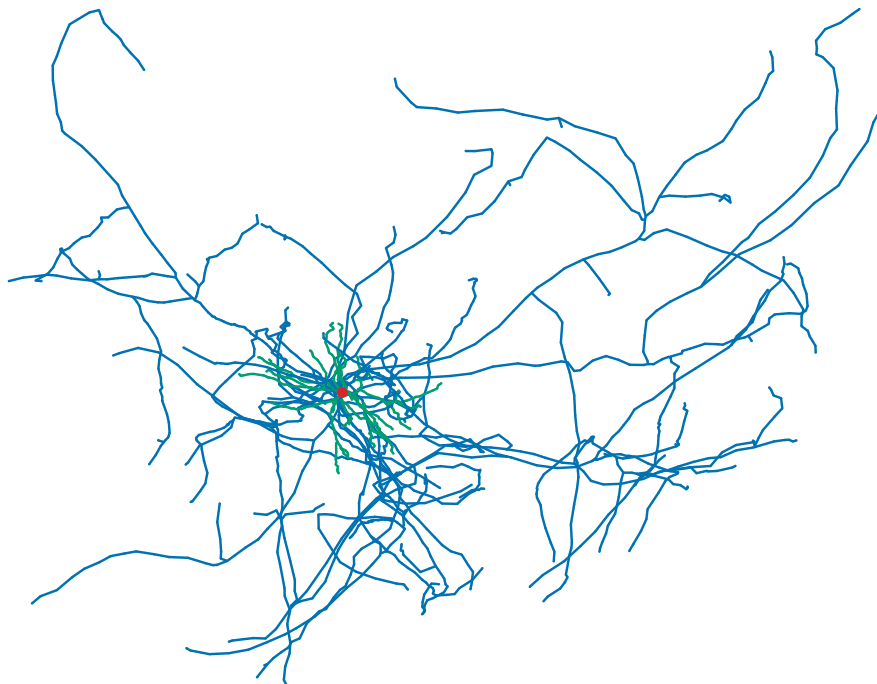
Pure Transformations

Every transformation returns a new cell. The input remains unchanged. Results include transform-report, mapping, and lineage; the report records source/result fingerprints, node counts, removed and inserted IDs, cable-length change, and any guaranteed deviation bound.

VIII.1 SWC-Compatible Geometry

`#translate`, `#rotate`, `#uniform-scale`, `#reflect`, `#center-morphology`, and `#pca-align` preserve circular SWC radius semantics. Translation, rotation, reflection, centering, and principal alignment keep radii unchanged. Uniform scaling multiplies coordinates and radii by the same positive factor. Topology, node IDs, kinds, and parent links are preserved.

```
#let centered = swc.center-morphology(cell, by: "centroid")
#let frame = swc.principal-frame(centered, origin: "centroid")
#let aligned = swc.pca-align(centered, frame: frame)
#let rotated = swc.rotate(
  aligned,
  axis: (0, 0, 1),
  angle: 15deg,
)
```



Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

VIII.2 General Affine Geometry

`#affine-transform` is deliberately separate. A non-similarity affine map turns a circular cross-section into an ellipse, which SWC cannot encode. The caller must choose `radius-policy`: "preserve" or "volume-equivalent"; the transform report records the matrix, determinant, applied radius scale, policy, and `radius-representation-lossy: true`. Singular and non-finite transforms are rejected.

General affine output is a centerline transform with an explicitly approximate radius representation. Do not describe it as an exact geometric transform of neuronal volume or membrane surface.

VIII.3 Selection and Extraction

`#select-nodes` and `#select-kinds` return the induced forest over retained original edges. They never invent an edge across a removed node. `#subtree` extracts a node and every descendant. `#path` extracts the unique undirected route between two nodes in one component.

```
#let selected = swc.select-kinds(cell, kinds: (3, 4))
#let branch = swc.subtree(cell, node: 12)
#let route = swc.path(cell, start: 4, end: 11)
```

Selected dendrites	477 nodes
Subtree at node 12	42 nodes
Path 4 to 11	8 nodes

Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

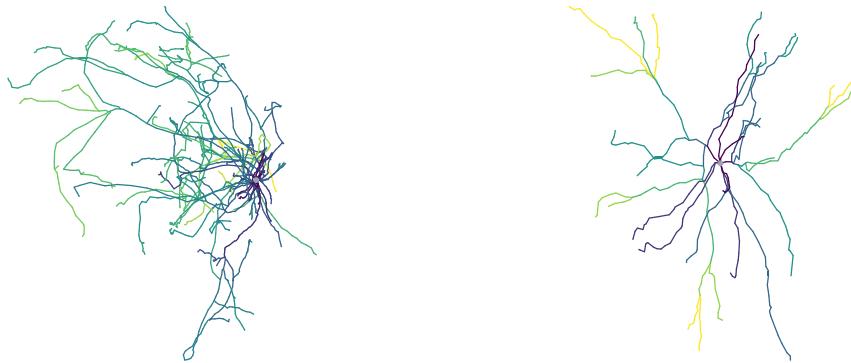
VIII.4 Rerooting and Pruning

`#reroot` reverses the parent chain so the requested node becomes the root of its component. `#prune` removes every node of the listed kinds and each removed node's complete descendant subtree.

```
#let metrics = swc.analyze(cell)
#let dendrites = swc.prune(cell, kinds: (2,))
#let dendrite-metrics = swc.analyze(dendrites)

#grid(
  columns: 2,
  gutter: 5mm,
  swc.render(cell, color-by: metrics.branch_order),
```

```
swc.render(dendrites, color-by: dendrite-metrics.branch_order),  
)
```



Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

VIII.5 Simplification

`#simplify` applies topology-preserving 3D Ramer-Douglas-Peucker simplification independently between mandatory section points. Roots, branch points, terminals, protected IDs, soma nodes, and type changes are retained according to the options. The report's `guaranteed-max-deviation` is the requested tolerance.

Simplification is also available as the display-only (`display-tolerance`) option of `#render`. Display simplification never changes the source cell, analysis results, or export; requested native annotations, CeTZ labels, and explicit node anchors are protected automatically.

VIII.6 Resampling

`#resample` inserts equal-arc-length samples within non-soma, same-type topological sections. Roots, soma nodes, branch points, terminals, type boundaries, and protected IDs retain their original IDs. Radius interpolation is linear in arc length, and every inserted node receives proximal/distal lineage with a distal fraction.

```
#let sampled = swc.resample(cell, step: 5)

#table(
  columns: 2,
  [Source nodes], [#cell.node-count],
  [Result nodes], [#sampled.node-count],
  [Inserted nodes], [#sampled.lineage.len()],
)
```

Source nodes	2172
Result nodes	18175
Inserted nodes	17919
Cable-length change	-296.80316729

Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

VIII.7 Deterministic Export

`#export-swc` emits canonical SWC with a deterministic header, topological row order, and sequential IDs. Single-root results satisfy `incf-strict`; forests retain every root and must be reloaded with `permissive` unless they are connected separately by the caller.

Part IX

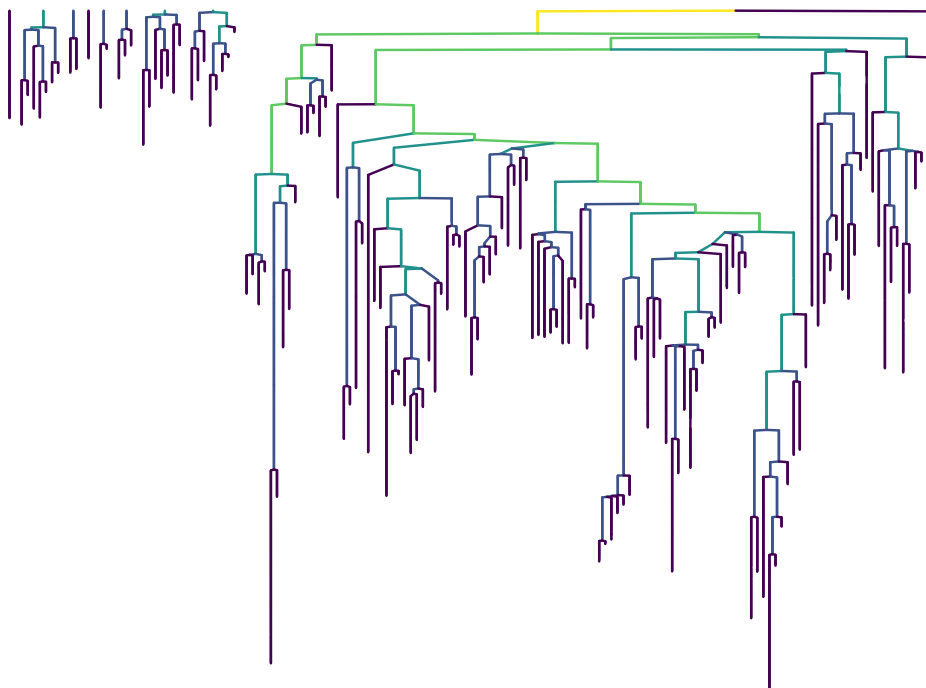
Topology Visualization

`#render-tree` draws the selected rooted forest independently of physical projection. Horizontal placement follows stable input/child order and leaf allocation. Vertical depth is one of topological edge depth, root path length, or Euclidean radial distance from each selected arbor root.

Strahler order and centrifugal branch order are discrete attributes, not interchangeable continuous distances. They belong in `color-by`; they are deliberately not accepted as depth modes.

```
#let metrics = swc.analyze(cell)

#swc.render-tree(
  cell,
  depth: "path-length",
  color-by: metrics.strahler_order,
)
```



Data: Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc NeuroMorpho.Org record 62495 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

The tree report names its depth rule explicitly. The renderer uses the same query layer and node-field fingerprint checks as physical rendering, and its returned node anchors can be used with the same CeTZ leader-label API.

Part X

Topological Morphology Descriptor

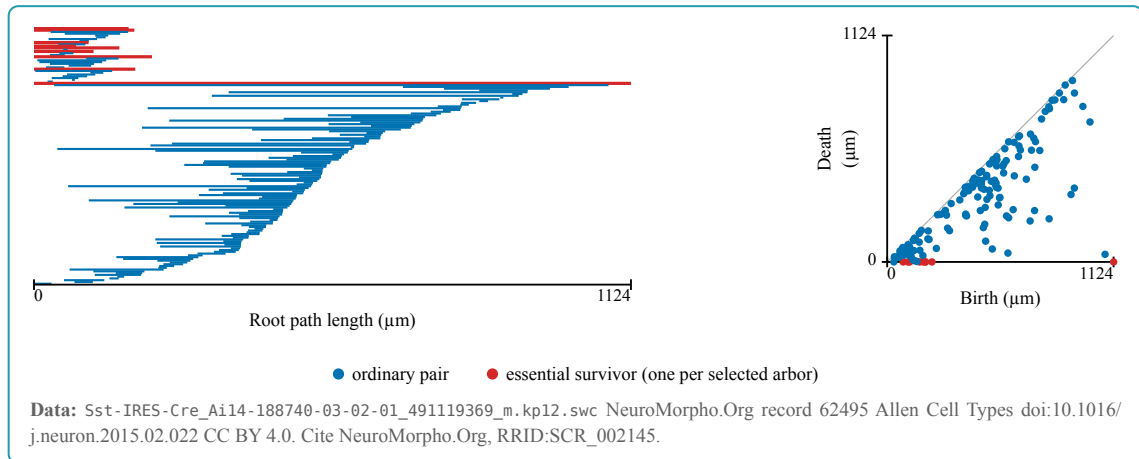
`#tmd` computes terminal-to-merge persistence pairs with the elder rule of the Topological Morphology Descriptor. `filtration`: "root-path-length" is monotone away from each selected arbor root, which has path distance zero; center is therefore inapplicable and rejected. "radial-distance" uses Euclidean distance from center: "soma" (the default) or center: "root" and can be non-monotone along a traced branch.

For each merge, the component with the larger terminal filtration survives; an exact tie is resolved by the lower terminal node ID. One essential survivor is retained per selected arbor. Every pair stores birth, death, absolute persistence, sorted display endpoints, terminal, merge, arbor root, essential status, and a `non_monotone` flag.

The returned pair array, and therefore barcode rows, is deterministic: arbor-root ID ascending, essential survivor first, birth descending, death descending, terminal-node ID ascending, then merge-node ID ascending. The exact rule is also stored in `provenance.pair_order`; the original TMD paper otherwise treats the barcode's vertical order as arbitrary.

A radial terminal may lie closer to the origin than an upstream merge. Axodendron reports such pairs through `non-monotone-pair-count` and preserves signed birth/death ordering instead of clamping, discarding, or silently redefining the filtration. Treat these as disclosed terminal-branch TMD pairs, not as evidence that radial distance was monotone.

```
#let descriptor = swc.tmd(  
  cell,  
  filtration: "root-path-length",  
)  
#let scale = swc.persistence-scale(descriptor)  
  
#grid(  
  columns: 2,  
  swc.persistence-barcode(descriptor, scale: scale),  
  swc.persistence-diagram(descriptor, scale: scale),  
)  
#swc.persistence-legend()
```



Birth is horizontal and death vertical, matching the original TMD publication. Both plots use the same explicit physical scale, label it in the morphology's units, and apply blue to ordinary pairs and red only to the essential survivor retained for each selected arbor. Native renderers inset endpoint strokes and points from their clipped bounds. Essential bars remain finite root-to-terminal intervals rather than infinity. These conventions are recorded in provenance and must be matched across TMD implementations.

Part XI

Population Features

`#population` retains named morphologies, while `#feature-table` evaluates a common query and feature specification for every row. The primary result is Typst-native arrays and dictionaries. `#feature-table-csv` is a separate text export; missing CSV cells are empty, so the Typst result remains the authoritative source for missing reasons.

Morphology scalars must not specify an aggregate. Node, section, and bifurcation fields must specify one. Vector and box morphology metrics require one explicit component: `x`, `y`, or `z` for a centroid; `major`, `middle`, or `minor` for principal extents; and `min-x` through `span-z` for a bounding box. This avoids silently flattening structured scientific results. Field aggregation is strict by default: if any selected entity is undefined, the cell is missing with reason `partial-field`. `missing-policy: "omit"` is an explicit instruction to aggregate only defined entities. Columns are rejected if their resolved metric definition, parameters, units, support, aggregate, or component differ across rows.

```
#let features = swc.feature-table(
  cells,
  columns: (
    swc.feature-column("fractional-anisotropy", name: "FA"),
    swc.feature-column("local-bifurcation-angle",
      aggregate: "mean", missing-policy: "omit"),
    swc.feature-column("centroid", component: "x"),
  ),
)
```

Morphology	FA	Mean local angle	Centroid x
AA0109	0.572	70.957	-573.492
Nr5a1	0.948	76.293	-2.826
Sst	0.69	87.117	-55.991

Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

Data: Nr5a1-Cre_Ai14-187777-05-02-01_491392821_m.kp1.swc NeuroMorpho.Org record 62390 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

Data: Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc NeuroMorpho.Org record 62495 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

Summaries report valid and missing row counts, mean, median, minimum, maximum, and both sample and population variance. No inferential statistic, imputation, unit conversion, or correction for repeated measurements is performed automatically.

Part XII

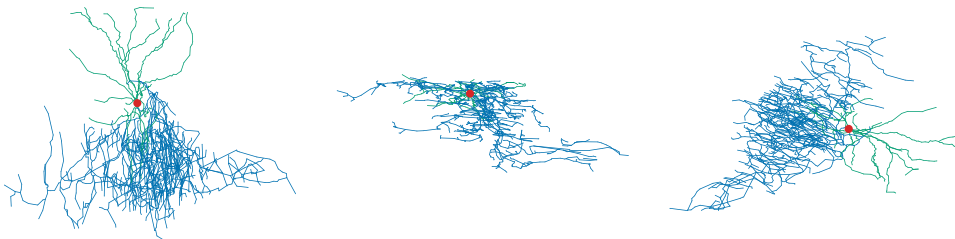
Rendering

#render produces a compact deterministic SVG in WASM and places it in a Typst block. Labels, markers, legends, color bars, and scale bars are then composed as native Typst content.

XII.1 Projections

Named projections are "xy", "xz", and "yz". An arbitrary orthographic camera is a dictionary containing direction and up vectors. The vectors must be finite, non-zero, and non-collinear.

```
#grid(  
  columns: 3,  
  gutter: 4mm,  
  swc.render(cell, projection: "xy"),  
  swc.render(cell, projection: "xz"),  
  swc.render(  
    cell,  
    projection: (  
      direction: (x: 1, y: 1, z: 1),  
      up: (x: 0, y: 0, z: 1),  
    ),  
  ),  
)
```



Data: Vipr2-IRES2-Cre_Ai14-310513-05-02-01_637021223_m.CNG.swc NeuroMorpho.Org record 102520 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

XII.2 Geometry and Radius Modes

geometry: "tapered" draws radius-aware frusta with round distal joins. "skeleton" draws constant-width centerlines. radius-mode: "physical" maps projected radii linearly. "readable" applies `<radius-exponent>` and the screen-space `<minimum-radius>` and `<maximum-radius>` bounds.

```
#grid(
  columns: 2,
  gutter: 5mm,
  swc.render(cell, geometry: "tapered", radius-mode: "readable"),
  swc.render(cell, geometry: "skeleton"),
)
```



Data: Nr5a1-Cre_Ai14-187777-05-02-01_491392821_m.kp1.swc NeuroMorpho.Org record 62390 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

XII.3 Soma Modes

soma-mode: "equivalent-sphere" is the default and displays one body at the projected soma centroid with the representative encoded radius. "encoded" uses cylinder geometry for a geometrically valid three-point soma. "raw-points" displays every encoded type-1 node and edge. Display policy does not invent soma area or volume metrics.

XII.4 Type and Scalar Color

The default color-by: "type" mapping uses red #d62728 for soma, blue #0072b2 for axon, and green #009e73 for basal and apical dendrites. Types 5, 6, and 7 use #e69f00, #56b4e9, and #cc79a7; all other kinds use #4d4d4d.

Passing any other color string produces a uniform rendering. Passing a node field produces scalar color. Viridis is the default continuous map and Magma is also supported. Finite values are normalized linearly between `<minimum>` and `<maximum>`, inferred from the field when omitted; values outside the range are clamped. Missing or non-finite values use neutral gray #9ca3af.

```
#let metrics = swc.analyze(cell)

#grid(
  columns: 2,
  gutter: 5mm,
  swc.render(cell, color-by: metrics.root_path_length),
  swc.render(
    cell,
    color-by: metrics.root_path_length,
    colormap: "magma",
  ),
)
```



Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

XII.5 Fitting and Aspect Ratio

Canvas fitting includes painted radii, outlines, and soma extent. It must not clip painted geometry inside `<padding>`. The Typst ratio `width / height` must exactly match `canvas-width / canvas-height`; Axodendron rejects a mismatch so overlays and physical scale bars remain calibrated.

`background: none` keeps the SVG transparent. Set `<outline-color>` to add a halo around segments and soma; `<outline-width>` has no effect when the color is none. Style strings accept a bounded safe subset of CSS color syntax and reject external URLs.

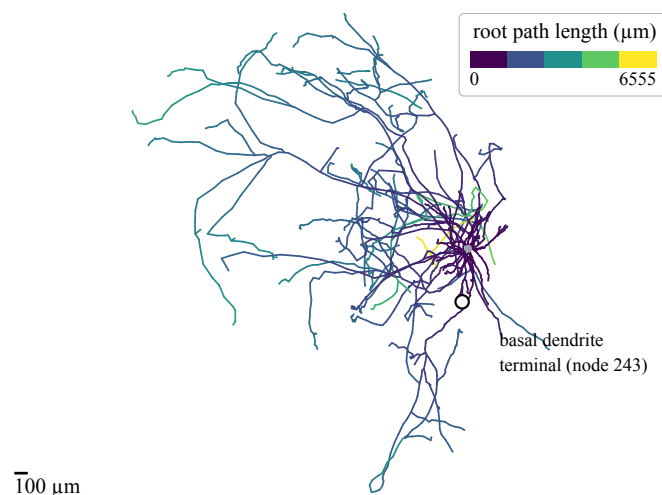
Part XIII

Native Typst Annotations

`#label` and `#marker` anchor native Typst content to projected node IDs; `{offset}` then shifts it by `x` and `y` lengths without changing the selected node. `#legend`, `#color-bar`, and `#scale-bar` place publication-oriented overlays inside the render block. These overlays remain Typst content rather than being flattened into SVG text; `{label-gap}` controls the vertical separation between a color-bar label and its palette strip.

```
#let metrics = swc.analyze(cell)
#let path-maximum = calc.ceil(calc.max(..metrics.root_path_length.values))

#swc.render(
  cell,
  color-by: metrics.root_path_length,
  minimum: 0,
  maximum: path-maximum,
  labels: (swc.label(
    node: 243, offset: (x: 14pt, y: 12pt),
    text(size: 7pt)[basal dendrite #linebreak() terminal (node 243)],
  )),
  markers: (swc.marker(node: 243)),
  color-bar: swc.color-bar(
    min: 0, max: path-maximum,
    label: [root path length (μm)], label-gap: 5pt, position: top + right,
  ),
  scale-bar: swc.scale-bar(value: 100),
)
```



Data: AA0109.CNG.swc NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

Unknown annotation IDs are errors by default. Set `(strict-node-ids)` to `false` only when optional annotations may legitimately target nodes removed by an earlier selection. Requested native labels, markers, CeTZ labels, and `(anchor-nodes)` are protected from display simplification.

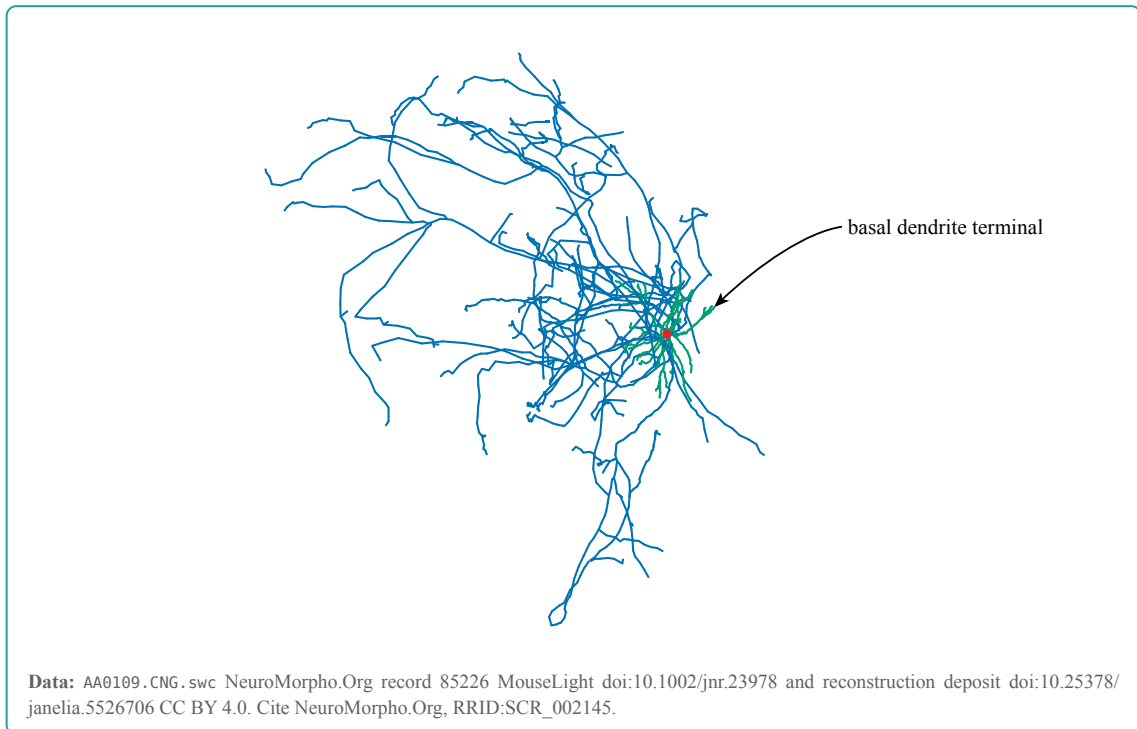
Label offsets and legend or color-bar positions are explicit layout choices rather than automatic collision-avoidance requests. Tune a node label with `offset: (x: ..., y: ...)`, choose a clear overlay corner with `(position)`, and use `(label-gap)` when the color-bar title needs more or less separation from the palette; this example labels SWC type-3 terminal node 243, marks the same node with a circle, uses a 14 pt rightward and 12 pt downward label shift, places the color bar in the clear top-right corner, and applies a 5 pt label gap.

XIII.1 CeTZ Leader Labels

CeTZ leader labels keep the text box away from morphology geometry while an arrow identifies the exact projected node. Import CeTZ separately and pass its module through `(cetz)` Axodendron has no mandatory CeTZ dependency and performs no second projection. `#render` requests and protects every `(cetz-labels)` target in the same WASM call, converts its final fitted screen coordinate to the CeTZ bottom-left coordinate system, registers it as a named CeTZ anchor, and composes the SVG, leader, and label in one canvas.

```
#import "@preview/axodendron:0.1.1" as swc
#import "@preview/cetz:0.5.2"

#swc.render(
  cell,
  width: 120mm,
  height: 90mm,
  cetz: cetz,
  cetz-labels: (swc.cetz-label(
    node: 447,
    offset: (x: 17mm, y: -9mm),
    controls: (
      (x: 12mm, y: -10mm),
      (x: 5mm, y: -5mm),
    ),
    text(size: 8pt)[basal dendrite terminal],
  )),
)
```



The `<offset>`, each `<via>` point, and each `<controls>` point are relative to the selected node in Typst screen coordinates: positive `x` moves right and positive `y` moves down. With `anchor: auto`, Axodendron places the label by the edge or corner facing the node. For a label beside the node, the leader itself starts at the mid-west or mid-east content anchor, so its initial `y` coordinate is the text's typographic vertical center rather than a corner. The default leader is straight; `<via>` inserts explicit straight segments. One `<controls>` point selects a quadratic CeTZ Bezier and two select a cubic Bezier. Axodendron never introduces curvature on its own. The default white fill and 2 pt padding keep morphology lines out of the text without a visible border. Set `<target-gap>` only when the arrow tip should stop short of the node.

A leader path is not an automatic obstacle-avoidance solver. A direct arrow can still cross an unrelated branch in a dense arbor, so use `<via>` for a deliberate polyline or `<controls>` for a deliberate curve and verify the final PDF. The two explicit control points in this example produce the displayed cubic Bezier. Node 447 is an encoded type-3 terminal with no child; the leader reaches that exact rendered sample without crossing the neighboring arbor.

For a custom CeTZ composition, request sparse coordinates with `anchor-nodes: (447, ...)` and `return-report: true`, retrieve a record with `#node-anchor`, or pass the result to `#cetz-annotate`. `x` and `y` in each returned record are lengths from the top-left of the base render block; `x-ratio` and `y-ratio` are normalized screen coordinates, and `screen-x` and `screen-y` retain the fitted SVG coordinates.

XIII.2 Render Reports

Set `<return-report>` to `true` to receive `body`, `dimensions`, `node-anchors`, `report`, `pixels-per-unit`, `source-node-count`, and `rendered-node-count`. The report records radius and soma

modes, counts of radii floored or capped, the simplified node count, and the protected overlay node count.

Field	Interpretation
pixels-per-unit	Projected SVG pixels per morphology coordinate unit after fitting and padding
source-node-count	Canonical nodes in the input cell
rendered-node-count	Nodes retained for SVG geometry after display-only simplification
node-anchors	Sparse projected-node records requested by labels, markers, CeTZ labels, or anchor-nodes; all rendered nodes when include-nodes is true
floored-radius-count	Rendered radii raised to minimum-radius
capped-radius-count	Rendered radii limited by maximum-radius or maximum-soma-radius
simplified-node-count	Source nodes omitted only from display geometry
overlay-node-count	Unique valid node IDs protected for native labels, markers, CeTZ labels, or explicit anchors

The report is disclosure metadata, not a transformed morphology. Analysis and export still use the input cell. For a reproducible publication figure, retain the projection, page and canvas dimensions, display tolerance, radius and soma modes, scalar range, palette, and the report alongside the input fingerprint.

Part XIV

Publication Figure Guidance

Use the type palette when compartment identity matters and a sequential map only for an ordered scalar. Do not use branch-order color without a color bar in a figure intended for quantitative interpretation. State the analysis domain and units in the caption.

Use radius-mode: "physical" only when literal projected thickness is legible at the final size. The default readable mode is more robust for overview figures, but its exponent and screen-space caps must be reported when thickness is interpreted.

Keep labels sparse and outside dense arbors. Add a light outline only when a morphology crosses a colored or photographic background. Verify the final PDF at publication size; deterministic SVG structure does not guarantee identical rasterization across font and PDF toolchains.

Display simplification, readable radius scaling, equivalent-sphere soma display, and projection are presentation operations. They never change the cell or scientific analysis, but a publication caption should disclose them when they affect interpretation.

Part XV

Error Model and Resource Limits

Plugin errors are structured before the Typst wrapper turns them into readable panics. Stable error families are:

Family	Meaning
PROTOCOL_*	Malformed CBOR or incompatible protocol
PAYLOAD_*	Incompatible, forged, or internally inconsistent morphology payload
SWC_*	Lexical, structural, or validation failure
SHOLL_*	Invalid center, radius, domain, or projection
MEASURE_*	Unknown metric, invalid parameter, selection, or request size
PRINCIPAL_FRAME_*	Invalid or degenerate spatial frame
CENTER_*	Invalid or ambiguous center selection or geometry
QUERY_*	Invalid topology selector or selection query
FIELD_TO_NODES_*	Invalid explicit field conversion
TMD_*	Invalid filtration center, selection, or soma geometry
POPULATION_*	Invalid population size, column count, or comparability
TRANSFORM_*	Invalid node, component, tolerance, step, or ID space
RENDER_*	Invalid canvas, style, scalar field, projection, or overlay
LIMIT_*	Explicit bounded-resource failure

Resource	Limit
SWC source / encoded request	64 MiB / 64 MiB
Morphology nodes	250,000
Decoded payload / encoded response	128 MiB / 128 MiB
Sholl radii	10,000
SVG result	64 MiB
WASM artifact	1.25 MiB
Source package bundle	4 MiB

Limits are checked before unbounded output allocation. The normal quality gate also exercises the full 250,000-node parser boundary, bounded resampling expansion, a 100,000-node performance case, two clean byte-identical WASM builds, and package import compilation.

Part XVI

API Reference

The public string version reports the embedded plugin version. The public swc dictionary mirrors every function below for selective dictionary imports, but dictionary-stored functions require parenthesized calls such as `(swc.analyze)(cell)`. Importing the package as a module and calling `swc.analyze(cell)` remains the recommended interface.

XVI.1 Loading and Inspection

#load(({source}), {profile}: "permissive", {fail-on-error}: true) → cell

Parse and validate SWC input.

Argument

{source} str | bytes | path

SWC text or bytes. Typst 0.15.0 and later may pass `path(...)`; Typst 0.14.x should pass `read(..., encoding: none)` output.

Argument

{profile}: "permissive" str

"permissive" or "incf-strict".

Argument

{fail-on-error}: true bool

Panic on validation errors when true; return an invalid diagnostic-bearing cell when false.

#from-text(({source}), {profile}: "permissive", {fail-on-error}: true) → cell

Parse SWC source already held in a string or bytes value. Arguments and validation behavior match **#load** except that path input is not read.

#diagnostics(({cell})) → array

Return the retained diagnostic dictionaries.

#metadata(({cell})) → dictionary

Return retained comments and recognized header fields without applying corrections.

XVI.2 Analysis

#analyze(({cell}), {domain}: "neurites", {section-boundaries}: "topology-and-type")

→ analysis

Compute the complete versioned morphometrics bundle.

Argument

{domain}: "neurites" str

"neurites" excludes soma nodes and soma-incident edges; "raw" includes the encoded graph.

Argument

(section-boundaries): "topology-and-type"

str

"topology-and-type" or "topology-only".

```
#sholl(
  {cell},
  {radii}: none,
  {center}: none,
  {center-node}: none,
  {domain}: "neurites",
  {projection}: none
)
```

→ dictionary

Compute exact 3D sphere intersections. Supplying {projection} switches to physical 2D projected intersections and is normally exposed through [#sholl-2d](#).

```
#sholl-2d(
  {cell},
  {radii}: none,
  {projection}: "xy",
  {center}: none,
  {center-node}: none,
  {domain}: "neurites"
)
```

→ dictionary

Compute exact 2D circle intersections after the requested orthographic projection.

[#available-metrics](#) → array

Return the metric registry with stable IDs, independent definition versions, support entities, units, summaries, reference URLs, and machine-readable parameter schemas. Each parameter record includes its name, value type, default, allowed choices or numerical bound, conditional applicability, and summary.

```
#metric(
  {id},
  {p}: none,
  {diameter-sampling}: none,
  {taper-quantity}: none,
  {taper-method}: none,
  {multifurcation}: none,
  {weighting}: none,
  {plane}: none
)
```

→ dictionary

Construct a parameterized metric specification. Supplied parameters that are unknown or irrelevant to the selected metric are rejected; accepted defaults and explicit values are serialized into the resolved metric identity.

```
#measure(
  {cell},
  {metrics}: none,
  {domain}: "neurites",
  {kinds}: (),
  {roots}: (),
  {nodes}: (),
  {section-boundaries}: "topology-and-type"
) → array
```

Compute one or more versioned metrics over a common induced-forest selection and return tagged MetricResult dictionaries.

```
#principal-frame(
  {cell},
  {origin}: "centroid",
  {weighting}: "cable-length",
  {domain}: "neurites",
  {kinds}: (),
  {roots}: (),
  {nodes}: (),
  {relative-tolerance}: 1e-10,
  {absolute-tolerance}: 1e-12
) → dictionary
```

Compute a deterministic, provenance-bearing principal coordinate frame and report rank or repeated-eigenvalue ambiguity.

```
#select-node-ids({cell}, {selector}: "all") → array
```

Return IDs matching all, roots, branch-points, terminals, soma, or an order-selector dictionary under the common query arguments. branch-points, terminals, and soma-nodes are convenience wrappers.

```
#branch-order({exact}: none, {min}: none, {max}: none) → dictionary
```

Construct an exact or inclusive-range centrifugal branch-order selector. #branch-order-nodes applies it directly to one morphology.

```
#strahler-order({exact}: none, {min}: none, {max}: none) → dictionary
```

Construct an exact or inclusive-range Strahler-order selector. #strahler-order-nodes applies it directly to one morphology.

```
#branch-order-nodes({cell}, {exact}: none, {min}: none, {max}: none) → array
```

Return IDs matching a centrifugal branch-order constraint under the common query arguments.

```
#strahler-order-nodes({cell}, {exact}: none, {min}: none, {max}: none) → array
```

Return IDs matching a Strahler-order constraint under the common query arguments.

#field-to-nodes(**{cell}**, **{field}**: **none**, **{placement}**: **none**, **{reducer}**: **"error"**) → **dictionary**

Explicitly project a section or bifurcation field to nodes. Valid placements are section proximal/distal/broadcast and bifurcation branch/children; reducers are error, mean, minimum, maximum, or sum.

#tmd(**{cell}**, **{filtration}**: **"radial-distance"**, **{center}**: **auto**) → **dictionary**

Compute versioned terminal–merge TMD pairs under radial-distance or root-path-length filtration. Radial distance defaults to the soma and accepts center: "soma" or center: "root"; root-path length is measured from each selected arbor root and rejects a center. The common domain/kind/root/node selection arguments are also accepted.

#persistence-scale(**{descriptor}**, **{min}**: **none**, **{max}**: **none**) → **dictionary**

Resolve a physical scale that can be passed to both persistence plots. Explicit bounds must contain every birth and death value.

#persistence-barcode(
{descriptor},
{width}: **226.77pt**,
{height}: **198.43pt**,
{row-height}: **none**,
{scale}: **none**

) → **content**

Render every TMD pair in provenance-defined row order with a labeled filtration axis. The default distributes rows within **{height}** an explicit **{row-height}** requests fixed spacing and determines total height.

#persistence-diagram(**{descriptor}**, **{size}**: **198.43pt**, **{scale}**: **none**, **{radius}**: **1.4pt**) → **content**

Render TMD birth/death points with Birth on the horizontal axis, Death on the vertical axis, and physical units on both axes.

#persistence-legend → **content**

Render the shared plot key: blue for ordinary pairs and red for the essential survivor retained once per selected arbor.

XVI.3 Population Tables

#population-entry(**{id}**, **{cell}**: **none**) → **dictionary**

Construct one named population row.

#population(**{entries}**) → **array**

Validate a Typst array of population entries without performing analysis.

```
#feature-column(
  {metric},
  {name}: none,
  {aggregate}: none,
  {component}: none,
  {missing-policy}: "strict"
) → dictionary
```

Construct a comparable scalar feature column. Structured morphology metrics require an explicit component; field aggregates are mean, median, minimum, maximum, sum, sample-variance, or population-variance; missing policy is strict or explicit omit.

```
#feature-table({population}, {columns}: none) → dictionary
```

Evaluate common metric/query definitions, enforce parameter and unit compatibility, retain missing reasons, and compute descriptive summaries.

```
#feature-table-csv({table}) → str
```

Serialize the scalar table as RFC 4180-style CSV text without performing file I/O.

XVI.4 Selection and Transformations

```
#translate({cell}, {offset}: none) → cell
```

Translate coordinates by a three-dimensional morphology-unit offset.

```
#rotate({cell}, {axis}: none, {angle}: none, {center}: (0, 0, 0)) → cell
```

Apply a right-handed axis-angle rotation; numeric angles are radians and Typst angle values are accepted directly.

```
#uniform-scale({cell}, {factor}: none, {center}: (0, 0, 0)) → cell
```

Apply one positive factor to coordinates and radii about a fixed center.

```
#reflect({cell}, {normal}: none, {point}: (0, 0, 0)) → cell
```

Reflect coordinates across a plane while preserving circular radii.

```
#center-morphology({cell}, {by}: "soma", {target}: (0, 0, 0)) → cell
```

Translate a soma center or selected centroid to a target coordinate.

```
#pca-align({cell}, {frame}: none, {allow-degenerate}: false) → cell
```

Express coordinates in a matching principal frame; ambiguous frames require an explicit opt-in.

```
#affine-transform({cell}, {matrix}: none, {translation}: (0, 0, 0), {radius-policy}: "preserve") → cell
```

Apply an invertible general affine centerline transform with explicit lossy SWC radius policy.

```
#select-nodes({cell}, {nodes}: none) → cell
```

Select exactly the listed node IDs as an induced forest.

```
#select-kinds({cell}, {kinds}: none) → cell
```

Select nodes with the listed SWC kinds as an induced forest.

```
#subtree({cell}, {node}: none) → cell
```

Extract a node and all descendants.

#path(**{cell}**, **{start}**: **none**, **{end}**: **none**) → **cell**

Extract the unique undirected path between two nodes in one component.

#reroot(**{cell}**, **{node}**: **none**) → **cell**

Reverse a component's parent chain so **{node}** becomes its root.

#prune(**{cell}**, **{kinds}**: **none**) → **cell**

Remove listed SWC kinds and their complete descendant subtrees.

#simplify(
 {cell},
 {tolerance}: **none**,
 {preserve-type-changes}: **true**,
 {preserve-soma}: **true**,
 {protected-nodes}: **()**
) → **cell**

Apply topology-preserving 3D Ramer-Douglas-Peucker simplification.

#resample(**{cell}**, **{step}**: **none**, **{protected-nodes}**: **()**) → **cell**

Resample eligible sections at equal arc-length spacing and return interpolation lineage.

#export-swc(**{cell}**) → **str**

Return deterministic canonical SWC text with sequential IDs and topological row order.

XVI.5 Annotation Builders

#label(**{body}**, **{node}**: **none**, **{offset}**: **(x: 4pt, y: -4pt)**) → **annotation**

Construct a Typst-native label anchored at a node. **{offset}** contains Typst x and y lengths applied after projection; positive x moves right and positive y moves down.

#marker(
 {node}: **none**,
 {body}: **none**,
 {offset}: **(x: 0pt, y: 0pt)**,
 {size}: **5pt**,
 {fill}: **luma(100%)**,
 {stroke}: **0.8pt + luma(0%)**
) → **annotation**

Construct a node marker. When **{body}** is omitted, a circle is drawn.

```
#cetz-label(
  {body},
  {node}: none,
  {offset}: (x: 45.35pt, y: -28.35pt),
  {via}: (),
  {controls}: (),
  {anchor}: auto,
  {padding}: 2pt,
  {fill}: luma(100%),
  {label-stroke}: none,
  {arrow-stroke}: 0.7pt + luma(0%),
  {arrow-fill}: luma(0%),
  {mark}: "stealth",
  {mark-scale}: 0.7,
  {target-gap}: 0pt
) → annotation
```

Construct an optional CeTZ leader label. `{offset}`, every `{via}` dictionary, and every `{controls}` dictionary contain Typst x and y lengths relative to the node; positive x moves right and positive y moves down. `{anchor}` accepts `auto` or a CeTZ content-anchor string for label placement. A side leader starts at the text's typographic vertical center. With no `{controls}` it is a straight line or a `{via}` polyline; one control selects a quadratic CeTZ Bezier and two select a cubic Bezier. `{controls}` and `{via}` cannot be combined. `{padding}` creates space around the text, `{fill}` and `{label-stroke}` style its rectangular background, and the remaining arguments style or shorten the leader and arrow tip.

```
#node-anchor({render-result}, {node}: none) → dictionary
```

Retrieve one projected-node record from a render result. Request the ID with `{anchor-nodes}`, a native annotation, or a CeTZ label and set `{return-report}` to `true`.

```
#cetz-annotate(
  {render-result},
  {cetz}: none,
  {labels}: (),
  {length}: 1pt,
  {strict}: true
) → content
```

Compose values returned by `#cetz-label` over an existing render result. This two-stage form is useful when the same projected coordinates also drive caller-owned CeTZ elements; the one-stage `{cetz-labels}` renderer argument is preferred for ordinary leader labels.

```
#legend({entries}: none, {position}: right + top, {inset}: 8pt) → dictionary
```

Construct a compact categorical legend. Each entry has `label` and `color`.


```
#color-bar(  
  {min}: none,  
  {max}: none,  
  {label}: none,  
  {label-gap}: 4pt,  
  {colormap}: "viridis",  
  {position}: right + bottom,  
  {inset}: 8pt  
) → dictionary
```

Construct a scalar color bar using the renderer's named palette. {min} and {max} are aligned to the exact left and right ends of the palette strip. {label-gap} is the non-negative vertical space between {label} and the palette strip.

```
#scale-bar({value}: none, {label}: none, {inset}: 8pt, {stroke}: 1pt) → dictionary
```

Construct a physical scale bar. {value} is expressed in cell.units.

XVI.6 Renderer

```
#render(
    {cell},
    {projection}: "xy",
    {color-by}: "type",
    {colormap}: "viridis",
    {minimum}: none,
    {maximum}: none,
    {width}: 340.16pt,
    {height}: 255.12pt,
    {geometry}: "tapered",
    {radius-mode}: "readable",
    {soma-mode}: "equivalent-sphere",
    {canvas-width}: 800,
    {canvas-height}: 600,
    {padding}: 24,
    {stroke-width}: 2,
    {minimum-radius}: 1,
    {maximum-radius}: 18,
    {maximum-soma-radius}: 96,
    {radius-scale}: 1,
    {radius-exponent}: 0.5,
    {soma-scale}: 1,
    {background}: none,
    {outline-color}: none,
    {outline-width}: 1,
    {display-tolerance}: none,
    {include-nodes}: false,
    {anchor-nodes}: (),
    {labels}: (),
    {markers}: (),
    {cetz}: none,
    {cetz-labels}: (),
    {legend}: none,
    {color-bar}: none,
    {scale-bar}: none,
    {strict-node-ids}: true,
    {return-report}: false
```

) → **content**

Render deterministic SVG and compose optional native Typst overlays.

Argument

{projection}: "xy"

str | dictionary

"xy", "xz", "yz", or (direction:, up:).

Argument

`<color-by>: "type"``str` | `node-field`

"type", a safe CSS color string, or a fingerprint-bound scalar node field.

Argument

`<colormap>: "viridis"``str`

"viridis" or "magma" for scalar fields.

Argument

`<minimum>: none``float` | `int` | `none`

Scalar range minimum; inferred from finite field values when omitted.

Argument

`<maximum>: none``float` | `int` | `none`

Scalar range maximum; inferred from finite field values when omitted.

Argument

`<width>: 340.16pt``length`

Typst output width. Its ratio with `<height>` must match the numeric canvas ratio.

Argument

`<height>: 255.12pt``length`

Typst output height.

Argument

`<geometry>: "tapered"``str`

"tapered" or "skeleton".

Argument

`<radius-mode>: "readable"``str`

"readable" or "physical".

Argument

`<soma-mode>: "equivalent-sphere"``str`

"equivalent-sphere", "encoded", or "raw-points".

Argument

`<canvas-width>: 800``float` | `int`

Numeric SVG width in pixels. Its ratio with `<canvas-height>` must equal the Typst output ratio.

Argument

`(canvas-height): 600`

float | int

Numeric SVG height in pixels.

Argument

`(padding): 24`

float | int

Minimum fitted clearance in SVG pixels, including painted radii, soma extent, and outlines.

Argument

`(stroke-width): 2`

float | int

Skeleton stroke width and tapered-geometry distal join allowance in SVG pixels.

Argument

`(minimum-radius): 1`

float | int

Screen-space lower bound for rendered radii in readable mode.

Argument

`(maximum-radius): 18`

float | int

Screen-space upper bound for non-soma radii in readable mode.

Argument

`(maximum-soma-radius): 96`

float | int

Independent screen-space upper bound for soma radii in readable mode.

Argument

`(radius-scale): 1`

float | int

Multiplicative scale applied to neurite radii before screen-space bounds.

Argument

`(radius-exponent): 0.5`

float | int

Readable-mode exponent applied to projected radii; 1 preserves linear scaling before bounds.

Argument

`(soma-scale): 1`

float | int

Multiplicative scale applied to the displayed soma radius.

Argument

`(background): none`

str | none

Safe CSS color string for the SVG canvas, or none for transparency.

Argument

`{outline-color}: none` str none

Safe CSS color string for a segment and soma halo, or none to disable it.

Argument

`{outline-width}: 1` float intOutline width in SVG pixels; ignored when `{outline-color}` is none.

Argument

`{display-tolerance}: none` float int none

Optional morphology-unit tolerance for display-only topology-preserving simplification.

Argument

`{include-nodes}: false` boolRender every retained sample node as an SVG marker and include every projected node in `{node-anchors}`. Leave this off for ordinary figures and request sparse coordinates with `{anchor-nodes}`.

Argument

`{anchor-nodes}: ()` array

Integer node IDs whose final projected coordinates should be protected from display simplification and included in the render result without drawing markers.

Argument

`{labels}: ()` arraySequence returned by `#label`. Target IDs are protected from display simplification.

Argument

`{markers}: ()` arraySequence returned by `#marker`. Target IDs are protected from display simplification.

Argument

`{cetz}: none` function noneCaller-imported CeTZ module. Required only when `{cetz-labels}` is non-empty; Axodendron does not import CeTZ itself.

Argument

`{cetz-labels}: ()` arraySequence returned by `#cetz-label`. Targets are protected, projected, registered as CeTZ anchors, and composed over the base render.

Argument	
<code>{legend}: none</code>	dictionary none
Overlay returned by <code>#legend</code> .	
Argument	
<code>{color-bar}: none</code>	dictionary none
Overlay returned by <code>#color-bar</code> .	
Argument	
<code>{scale-bar}: none</code>	dictionary none
Overlay returned by <code>#scale-bar</code> .	
Argument	
<code>{strict-node-ids}: true</code>	bool
Reject unknown native-label, marker, CeTZ-label, or explicit anchor IDs. If false, missing optional annotations are skipped.	
Argument	
<code>{return-report}: false</code>	bool
Return a dictionary containing body, base dimensions, projected node anchors, and fit, simplification, radius, and overlay metadata instead of content alone.	

```
#render-tree(
  {cell},
  {depth}: "topological",
  {color-by}: "type",
  {domain}: "neurites",
  {width}: 340.16pt,
  {height}: 255.12pt,
  {return-report}: false
) → content
```

Render an abstract rooted forest. Depth is `topological`, `path-length`, or `radial-distance`; branch and Strahler orders are supplied as discrete color fields. The full function also accepts the common selection, canvas, node-anchor, and CeTZ-label arguments documented for physical rendering.

Part XVII

Result Schemas

XVII.1 Analysis Bundle

The analysis bundle contains `schema-version`, `definition-version`, `fingerprint`, `domain`, `summary`, `topology`, `sections`, `tortuosity`, `root-path-length`, `radial-distance`, `branch-order`, and `strahler-order`. The current definition version is `axodendron-morphometrics-1`.

The summary includes raw/domain node counts, edges, roots, components, branches, terminals, sections, total cable length, maximum path and radial distances, bounding box, type counts and metrics, per-arbor metrics, radius metrics, soma metrics and class, and units.

XVII.2 Metric Result

A `MetricResult` contains `schema_version`, `metric`, `selection`, `source`, `provenance`, `data`, and `missing`. `metric` separates `id`, `definition_version`, and resolved parameters. `source` contains morphology, topology, and selection fingerprints. `data.kind` is a tagged support type; its value contains aligned entity keys, values, and units. `missing` contains explicit entity/reason/detail records.

XVII.3 Principal Frame

A principal frame contains matching morphology and topology fingerprints, the resolved selection and weighting, origin mode and coordinate, centroid, three ordered axes and eigenvalues, min/max extents, rank, ambiguous-axis flags, and provenance containing definition version, covariance model, sign and handedness rules, tolerances, and selection fingerprint.

XVII.4 TMD Result

A TMD result contains schema and definition provenance, matching fingerprints, query, filtration, optional radial center, units, node-aligned filtration values, persistence pairs, and the non-monotone pair count. Definition version 2 makes each selected arbor root zero for root-path length and forbids a center for that filtration. `provenance.pair_order` records the deterministic barcode order. Essential and radial non-monotone conventions are explicit fields rather than implicit plotting behavior.

XVII.5 Feature Table

A feature table contains versioned column metadata, rows with morphology fingerprints and tagged value/missing cells, and descriptive summaries. Column metadata retains the metric descriptor, aggregate, structured component, missing policy, and units.

XVII.6 Transform Result

Transformed cells add `transform-report`, `mapping`, and `lineage`. `mapping` contains `old-id` and optional `new-id`; `lineage` contains `new-id`, `proximal-old-id`, `distal-old-id`, and `distal-fraction` for inserted samples. Geometry operations additionally record API class, affine matrix and translation, determinant, radius scale and policy, and whether SWC radius representation is lossy.

XVII.7 Render Result

With `return-report: true`, the renderer returns `body`, `width`, `height`, `canvas-width`, `canvas-height`, `node-anchors`, `report`, `pixels-per-unit`, `source-node-count`, and `rendered-node-count`. A node-anchor record contains `node`, top-left-relative `x` and `y` lengths, normalized `x-ratio` and `y-ratio`, SVG `screen-x` and `screen-y`, and projected depth. Use `body` as content and retain the result when a reproducible figure must disclose display simplification, radius clamping, or annotation coordinates.

Part XVIII

Limitations

- Axodendron renders orthographic views only; it does not provide perspective projection, lighting, or volumetric microscopy rendering.
- SWC expresses centerlines and sample radii, not membrane meshes, synapses, confidence intervals, or image registration. Axodendron does not infer information absent from the file.
- The default readable radius mode is a display policy, not a literal physical cross-section. Use physical mode and report it when width carries quantitative meaning.
- Equivalent-sphere soma display is a visual summary. Only supported soma encodings receive documented area and volume metrics.
- Scalar color is linear and supports Viridis or Magma. Axodendron does not automatically choose diverging, logarithmic, or categorical maps for a scientific hypothesis.
- Native and CeTZ labels use exact projected node anchors and explicit offsets; CeTZ via points define polylines and controls define Bezier curves, but collision-free placement is not automatic.
- Rendering tests guarantee deterministic SVG structure and broad real-data coverage. CI additionally hashes the exact decoded RGBA dimensions and pixels of a PNG baseline under Typst 0.14.2, so PNG metadata and compression do not create false regressions; this fixed-toolchain baseline does not claim identical raster output across unrelated PDF engines, fonts, or operating systems.

Part XIX

License, Dependencies, and Data Citations

Axodendron source is distributed under the MIT license. The compiled WASM includes `ciborium`, `serde`, `half`, `cfg-if`, `zerocopy`, and `wasm-minimal-protocol`; their selected licenses and exact versions are recorded in `THIRD_PARTY_NOTICES.md` and `wasm-plugin/Cargo.lock`. `CeTZ 0.5.2` is an optional LGPL-3.0-or-later example and documentation dependency supplied by the caller; its code is not embedded in Axodendron or `plugin.wasm`.

The four real SWC files used throughout this manual remain CC BY 4.0 material. Their per-file record links, archive names, original studies, and SHA-256 checksums are recorded in `THIRD_PARTY_NOTICES.md`. Every rendered occurrence in this manual also carries an adjacent attribution line.

When reusing a real-data figure, preserve its adjacent attribution, cite the original study, cite NeuroMorpho.Org with `RRID:SCR_002145`, and follow the NeuroMorpho.Org terms of use. The database citation used by this manual is Tecuatl C, Ljungquist B, Ascoli GA (2024), *Accelerating the continuous community sharing of digital neuromorphology data*, FASEB BioAdvances 6(7):207-221, [doi:10.1096/fba.2024-00048](https://doi.org/10.1096/fba.2024-00048)¹².

Metric fixtures are recomputed against NeuroM 4.0.5's `bifurcation`¹³ and `section`¹⁴ functions and L-Measure 5.2 revision 510 `Pk_classic`. The committed fixture records upstream callable, native units and values, Axodendron values, tolerance, compatibility class, and every known convention or precision difference. TMD provenance cites Kanari L et al. (2018), *A Topological Representation of Branching Neuronal Morphologies*, Neuroinformatics 16:3–13, [doi:10.1007/s12021-017-9341-1](https://doi.org/10.1007/s12021-017-9341-1)¹⁵. Diameter-power interpretation follows the recorded exponent rather than treating $p = \frac{3}{2}$ as a universal biological constant.

The detailed file-level notice is authoritative for bundled third-party material. This manual summarizes it for figure readers but does not replace `THIRD_PARTY_NOTICES.md`.

¹²<https://doi.org/10.1096/fba.2024-00048>

¹³https://neurom.readthedocs.io/en/stable/_neurom_build/neurom.features.bifurcation.html

¹⁴https://neurom.readthedocs.io/en/stable/_neurom_build/neurom.features.section.html

¹⁵<https://doi.org/10.1007/s12021-017-9341-1>

Part XX

Index

A

#affine-transform 38
analysis 34
#analyze 34
annotation 34
#available-metrics 35

B

#branch-order 36
#branch-order-nodes 36

C

cell 34
#center-morphology 38
#cetz-annotate 40
#cetz-label 40
#color-bar 41

D

#diagnostics 34

E

#export-swc 39

F

#feature-column 38
#feature-table 38
#feature-table-csv 38
#field-to-nodes 37
#from-text 34

L

#label 39
#legend 40
#load 34

M

#marker 39
#measure 36
#metadata 34
#metric 35

N

#node-anchor 40
node-field 34

P

#path 39
#pca-align 38
#persistence-barcode 37
#persistence-diagram 37
#persistence-legend 37
#persistence-scale 37
#population 37
#population-entry 37
#principal-frame 36
#prune 39

R

#reflect 38
#render 42
render-result 34
#render-tree 46
#reroot 39
#resample 39
#rotate 38

S

#scale-bar 41
#select-kinds 38
#select-node-ids 36
#select-nodes 38
#sholl 35

#sholl-2d 35
#simplify 39
#strahler-order 36
#strahler-order-nodes 36
#subtree 38

T

#tmd 37
#translate 38

U

#uniform-scale 38